To illustrate the practical application of MongoDB with Laravel, consider a product inventory system. Using MongoDB's schema flexibility, you can store products with varying attributes in a single collection. You can build a controller that handles storing, updating, and listing products. These operations are mapped to routes and views using Laravel's MVC pattern. For example, a form can submit data to a controller method that calls `Product::create()`, and a list view can iterate over `Product::all()` to display available products. This approach demonstrates how MongoDB can power real-world applications with dynamic data models.

12.9 MongoDB and Eloquent Relationships (Limitations)

MongoDB does not support joins in the same way as SQL databases. Instead, it encourages data modeling approaches such as embedding related documents or manually referencing them using IDs. While the Laravel MongoDB package offers basic support for relationships like `hasMany` and `belongsTo`, these are not as robust as traditional Eloquent relationships. For one-to-many relationships, you can store an array of embedded documents or a list of referenced IDs. It is important to design your data model carefully, balancing between embedding for performance and referencing for flexibility.

12.10 Exercise 36: CRUD Web App with MongoDB in Laravel 12

12.10.1 Description

In this hands-on lab, we will build a web application using Laravel 12 that performs Create, Read, Update, and Delete (CRUD) operations with MongoDB. We'll use the `jenssegers/laravel-mongodb` package as the MongoDB driver. The app will manage a collection of products, each having a name, price, and stock.

12.10.2 Objectives

- Install and configure MongoDB in a Laravel 12 application.
- Use `jenssegers/laravel-mongodb` to interact with MongoDB.
- Create a Product model and controller.
- Build Bootstrap-styled views to manage products (CRUD).